

Class Activity – 2

Scenario-01: Smart City Air Quality Monitoring

Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_excel("Scenario01.xlsx")
plt.figure(figsize=(8,6))
scatter = plt.scatter(
    data["X_Coordinate"],
    data["Y_Coordinate"],
    c=data["PM2.5"],
    cmap="YlOrRd",
    s=120,
    edgecolor="black"
)
plt.colorbar(scatter,label="PM2.5")
plt.title("Sequential Color Scale (PM2.5)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,6))
scatter = plt.scatter(
    data["X_Coordinate"],
    data["Y_Coordinate"],
    c=data["Temperature"],
    cmap="coolwarm",
    s=120,
    edgecolor="black"
```

```

)

plt.colorbar(scatter,label="Temperature (°C)")
plt.title("Diverging Color Scale (Temperature)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()

def pollution_level(pm):
    if pm < 50:
        return "Good"
    elif pm < 100:
        return "Moderate"
    else:
        return "High"

data["Pollution_Level"] = data["PM2.5"].apply(pollution_level)

colors = {
    "Good":"green",
    "Moderate":"orange",
    "High":"red"
}

plt.figure(figsize=(8,6))

for level in colors:
    subset = data[data["Pollution_Level"]==level]
    plt.scatter(
        subset["X_Coordinate"],
        subset["Y_Coordinate"],
        color=colors[level],
        label=level,
        s=120,
        edgecolor="black"
    )

```

```

)

plt.title("Qualitative Color Scale (Pollution Categories)")

plt.xlabel("X Coordinate")

plt.ylabel("Y Coordinate")

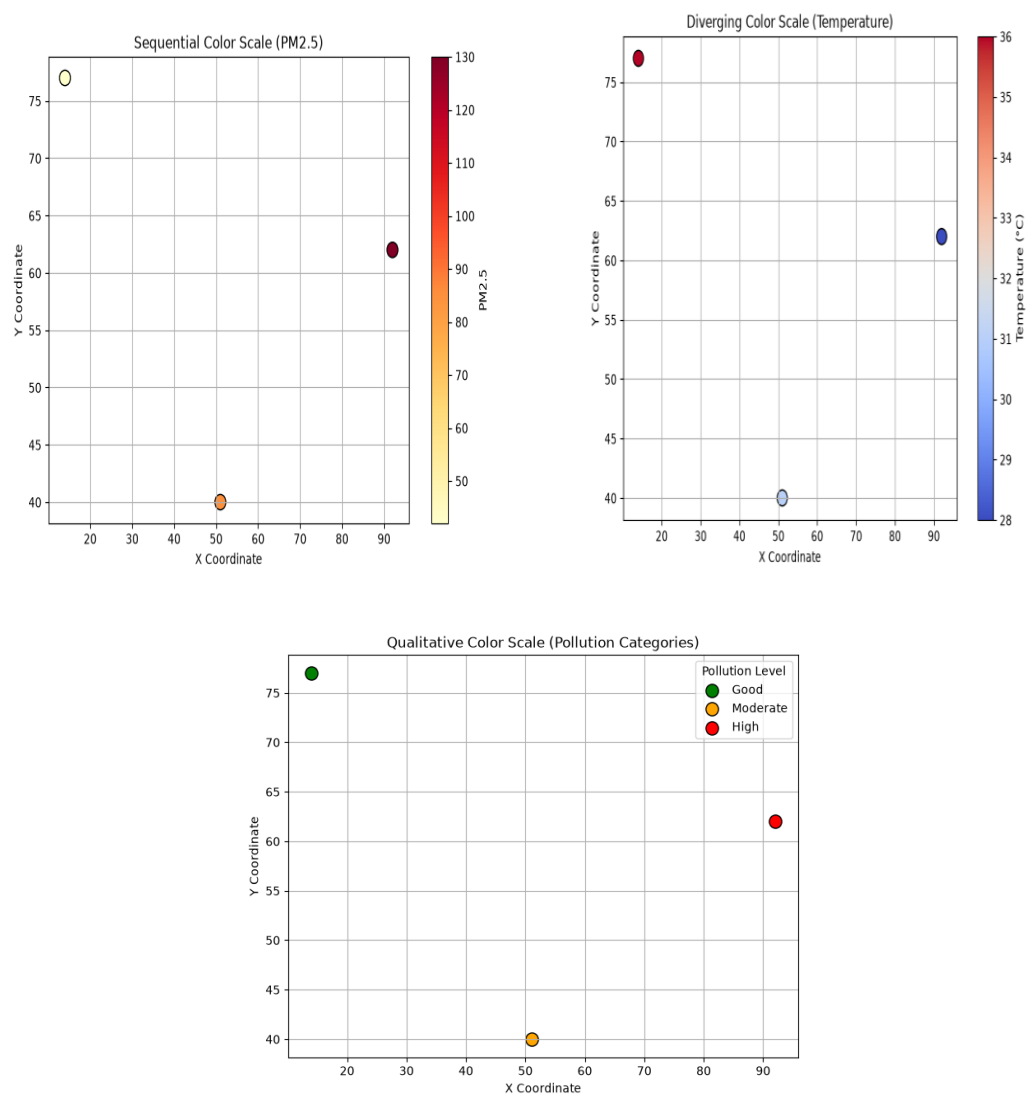
plt.legend(title="Pollution Level")

plt.grid(True)

plt.show()

```

Output:



R code:

```
library(ggplot2)
library(RColorBrewer)
# Create Dataset
air_data <- data.frame(
  X_Coordinate = c(10,20,30,40,50,60,70,80,90,100),
  Y_Coordinate = c(15,25,35,45,55,65,75,85,95,105),
  PM25 = c(25,45,60,80,95,110,130,150,170,190),
  NO2 = c(18,22,28,35,40,48,52,60,68,75),
  CO = c(0.6,0.8,1.0,1.2,1.5,1.8,2.0,2.3,2.5,2.8),
  Temperature = c(28,29,30,31,32,33,34,35,36,37),
  Humidity = c(70,68,66,64,62,60,58,56,54,52)
)
print(air_data)

# Sequential Palette
ggplot(air_data,
  aes(X_Coordinate, Y_Coordinate, color = PM25)) +
  geom_point(size = 5) +
  scale_color_gradient(low = "yellow", high = "red") +
  ggtitle("Sequential Palette - PM2.5") +
  theme_minimal()

# Diverging Palette
ggplot(air_data,
  aes(X_Coordinate, Y_Coordinate, color = Temperature)) +
  geom_point(size = 5) +
  scale_color_gradient2(low = "blue",
    mid = "white",
```

```

    high = "red",
    midpoint = mean(air_data$Temperature)) +
ggtitle("Diverging Palette - Temperature") +
theme_minimal()

# Qualitative Palette
air_data$Pollution_Level <- cut(
  air_data$PM25,
  breaks = c(0,50,100,200),
  labels = c("Good","Moderate","High")
)

ggplot(air_data,
  aes(X_Coordinate, Y_Coordinate, color = Pollution_Level)) +
  geom_point(size = 5) +
  scale_color_brewer(palette = "Set1") +
  ggtitle("Qualitative Palette") +
  theme_minimal()

```

Output:



Scenario-02: Earthquake Risk Analysis

Python Code:

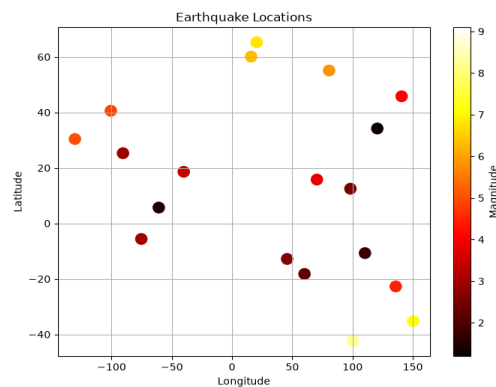
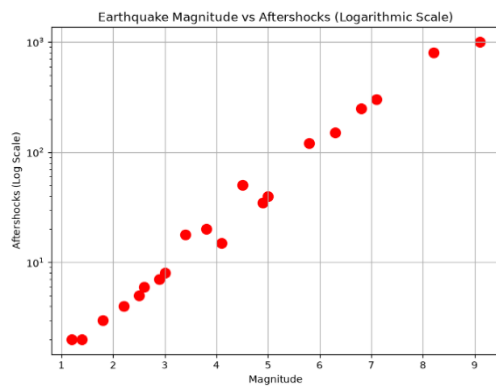
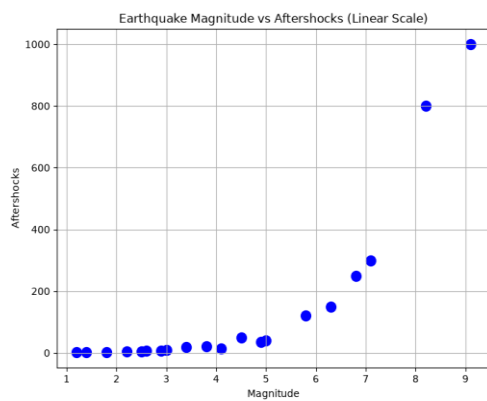
```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_excel("scenario02.xlsx")
plt.figure(figsize=(8,6))
plt.scatter(
    data["Magnitude"],
    data["Aftershocks"],
    color="blue",
    s=100
)
plt.title("Earthquake Magnitude vs Aftershocks (Linear Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershocks")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,6))
plt.scatter(
    data["Magnitude"],
    data["Aftershocks"],
    color="red",
    s=100
)
plt.yscale("log")
plt.title("Earthquake Magnitude vs Aftershocks (Logarithmic Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershocks (Log Scale)")
plt.grid(True)
plt.show()
```

```

plt.figure(figsize=(8,6))
scatter = plt.scatter(
    data["Longitude"],
    data["Latitude"],
    c=data["Magnitude"],
    cmap="hot",
    s=120
)
plt.colorbar(scatter, label="Magnitude")
plt.title("Earthquake Locations")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.grid(True)
plt.show()

```

Output:



R code:

```
library(ggplot2)

earthquake_data <- data.frame(
  Magnitude = c(2.1,2.8,3.4,4.2,4.8,5.5,6.2,6.9,7.5,8.1),
  Depth = c(15,30,45,60,90,120,180,250,350,500),
  Latitude = c(10,12,15,18,20,24,28,32,36,40),
  Longitude = c(72,74,76,78,80,82,84,86,88,90),
  Aftershock_Frequency = c(2000,1500,1000,600,300,150,80,40,20,10)
)

print(earthquake_data)

# Cartesian Plot
ggplot(earthquake_data,
  aes(Magnitude, Aftershock_Frequency)) +
  geom_point(color = "blue", size = 5) +
  ggtitle("Linear Scale") +
  theme_minimal()

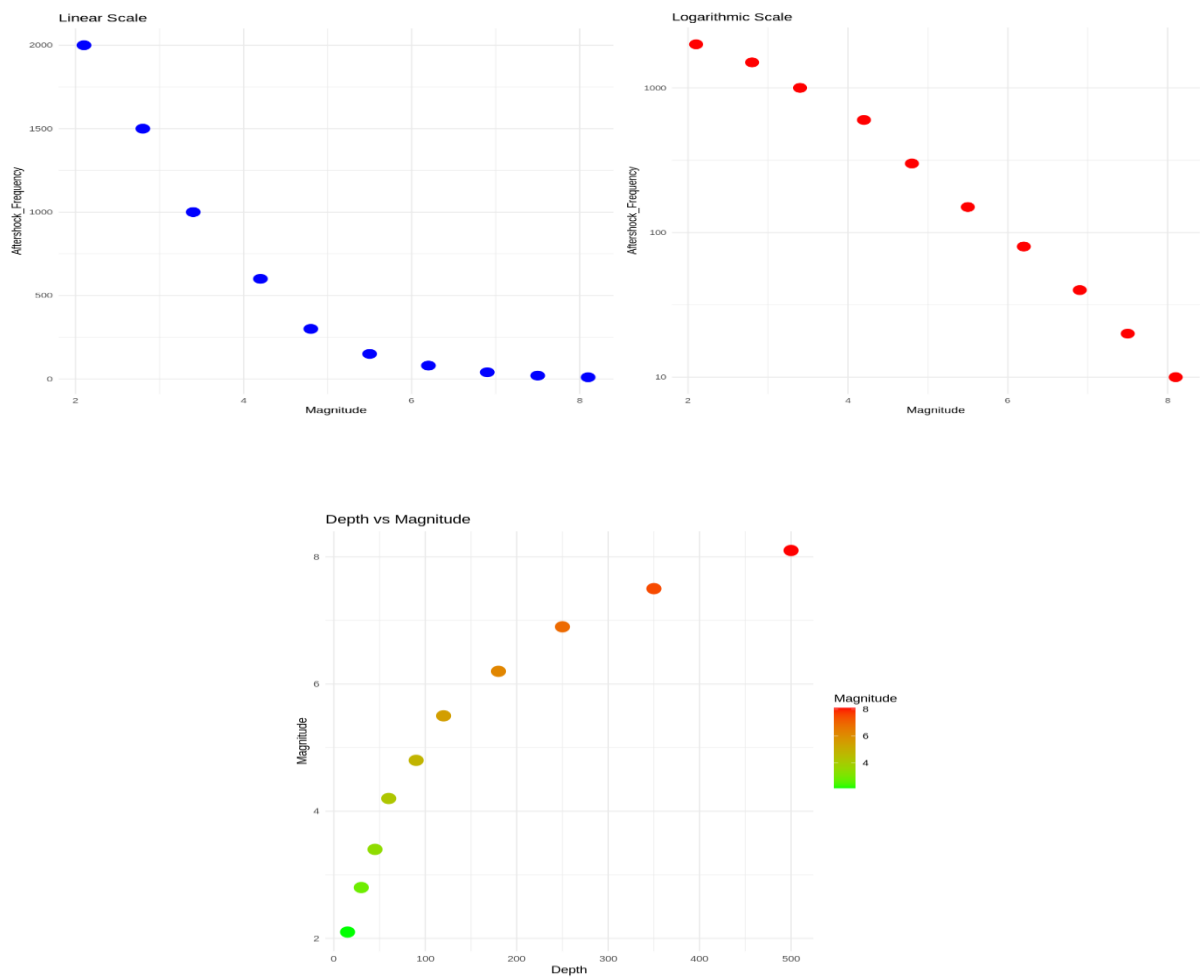
# Logarithmic Scale
ggplot(earthquake_data,
  aes(Magnitude, Aftershock_Frequency)) +
  geom_point(color = "red", size = 5) +
  scale_y_log10() +
  ggtitle("Logarithmic Scale") +
  theme_minimal()

# Earthquake Locations
ggplot(earthquake_data,
  aes(Longitude, Latitude, color = Magnitude)) +
  geom_point(size = 5) +
  scale_color_gradient(low = "yellow", high = "red") +
  ggtitle("Earthquake Locations") +
  theme_minimal()
```

```
# Depth vs Magnitude
```

```
ggplot(earthquake_data,  
       aes(Depth, Magnitude, color = Magnitude)) +  
  geom_point(size = 5) +  
  scale_color_gradient(low = "green", high = "red") +  
  ggtitle("Depth vs Magnitude") +  
  theme_minimal()
```

Output:



Scenario-03: Wind Direction Analysis for Airport Operations

Python Code:

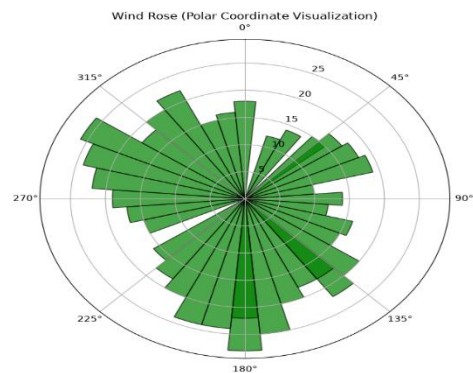
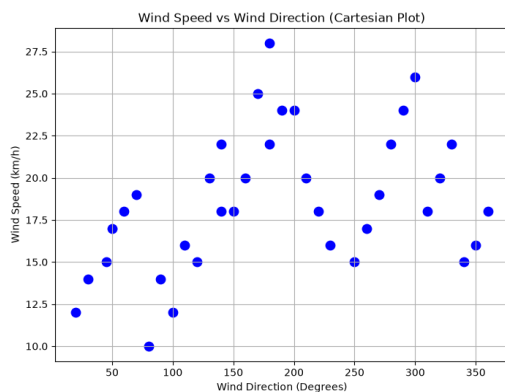
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
# Read Excel file
data = pd.read_excel("scenario3.xlsx")
plt.figure(figsize=(8,6))
plt.scatter(
    data["Wind_Direction"],
    data["Wind_Speed"],
    color="blue",
    s=80
)
plt.title("Wind Speed vs Wind Direction (Cartesian Plot)")
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed (km/h)")
plt.grid(True)
plt.show()
theta = np.deg2rad(data["Wind_Direction"])
r = data["Wind_Speed"]
plt.figure(figsize=(8,8))
ax = plt.subplot(111, projection='polar')
bars = ax.bar(
    theta,
    r,
    width=np.deg2rad(10),
    bottom=0,
    alpha=0.7,
    color="green",
```

```

    edgecolor="black"
)
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
plt.title("Wind Rose (Polar Coordinate Visualization)")
plt.show()

```

Output:



R code:

```

library(ggplot2)

wind_data <- data.frame(
  Wind_Speed = c(12,15,18,20,22,16,14,19,21,17,13,15),
  Wind_Direction = c(0,30,60,90,120,150,180,210,240,270,300,330)
)

print(wind_data)

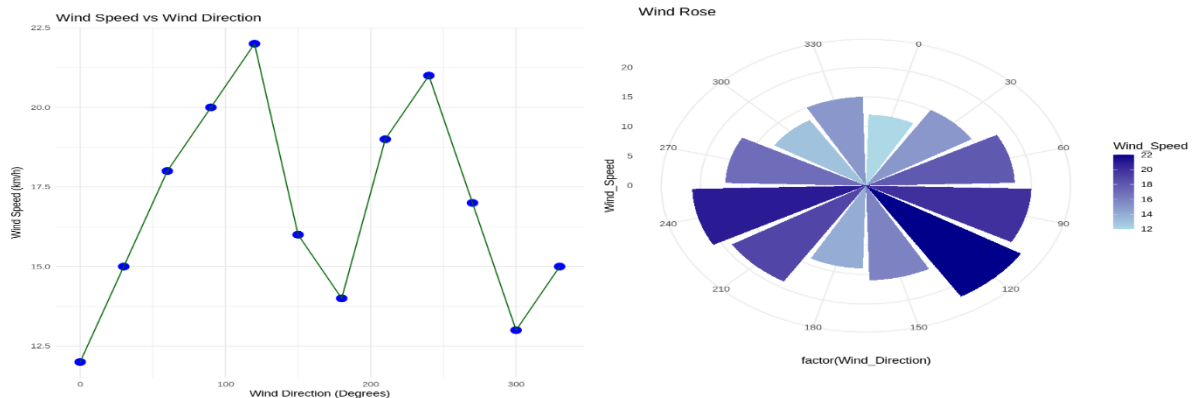
# Cartesian Plot
ggplot(wind_data,
  aes(Wind_Direction, Wind_Speed)) +
  geom_point(color="blue", size=4) +
  geom_line(color="darkgreen") +
  labs(title="Wind Speed vs Wind Direction",
    x="Wind Direction (Degrees)",
    y="Wind Speed (km/h)") +
  theme_minimal()

```

```
# Polar Plot (Wind Rose)

ggplot(wind_data,
       aes(factor(Wind_Direction), Wind_Speed, fill=Wind_Speed)) +
  geom_bar(stat="identity") +
  coord_polar() +
  scale_fill_gradient(low="lightblue", high="darkblue") +
  labs(title="Wind Rose") +
  theme_minimal()
```

Output:



Scenario-04: Hospital Patient Monitoring Dashboard

Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_excel("scenario4.xlsx")

def get_color(hr):
    if hr < 60 or hr > 120:
        return "red"    # Critical
    elif hr < 70 or hr > 100:
        return "orange" # Warning
    else:
        return "green"  # Normal
```

```

colors = data["Heart_Rate"].apply(get_color)

plt.figure(figsize=(14,8))

plt.subplot(2,3,1)

plt.bar(data["Patient_ID"], data["Heart_Rate"], color=colors)

plt.title("Heart Rate")

plt.xticks(rotation=90)

plt.subplot(2,3,2)

plt.bar(data["Patient_ID"],
        data["Oxygen_Saturation"],
        color=["red" if x<92 else "orange" if x<95 else "green"
               for x in data["Oxygen_Saturation"]])

plt.title("Oxygen Saturation")

plt.xticks(rotation=90)

plt.subplot(2,3,3)

plt.bar(data["Patient_ID"],
        data["Blood_Pressure"],
        color=["red" if x>140 else "orange" if x>130 else "green"
               for x in data["Blood_Pressure"]])

plt.title("Blood Pressure")

plt.xticks(rotation=90)

plt.subplot(2,3,4)

plt.bar(data["Patient_ID"],
        data["Respiration_Rate"],
        color=["red" if x>24 or x<12 else "orange" if x>20 else "green"
               for x in data["Respiration_Rate"]])

plt.title("Respiration Rate")

plt.xticks(rotation=90)

plt.subplot(2,3,5)

plt.bar(data["Patient_ID"],
        data["Temperature"],

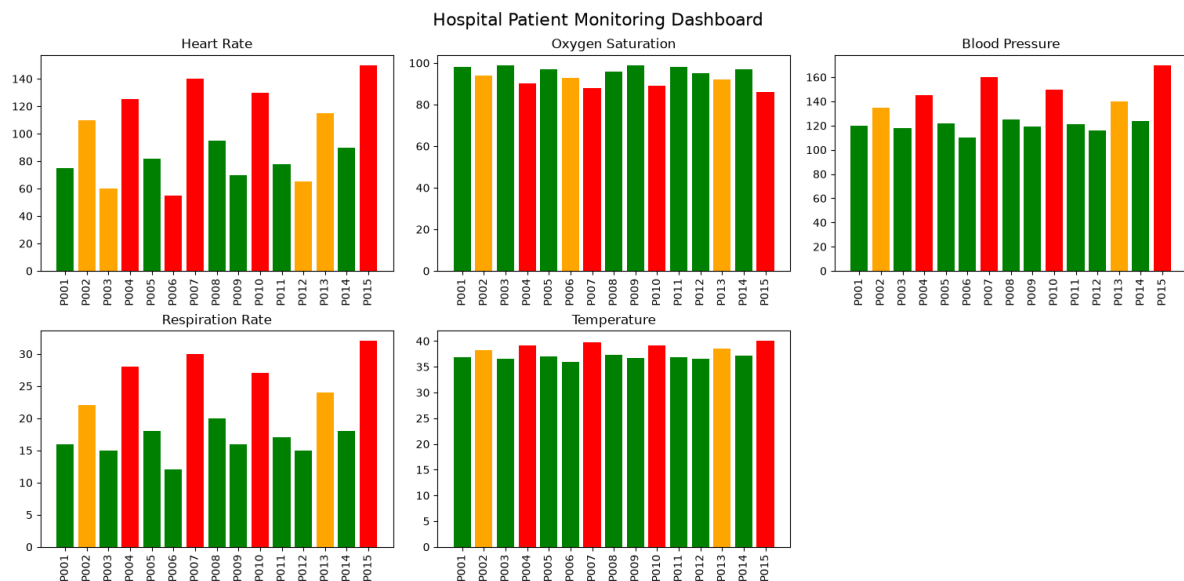
```

```

        color=["red" if x>=39 else "orange" if x>=37.5 else "green"
              for x in data["Temperature"]])
plt.title("Temperature")
plt.xticks(rotation=90)
plt.suptitle("Hospital Patient Monitoring Dashboard", fontsize=16)
plt.tight_layout()
plt.show()

```

Output:



R code:

```

library(ggplot2)

# Create Dataset
hospital_data <- data.frame(
  Patient_ID = paste("P",1:10,sep=""),
  Heart_Rate = c(72,85,95,110,130,78,88,125,100,65),
  Oxygen_Saturation = c(98,97,95,92,85,99,96,88,94,97),
  Blood_Pressure = c(120,125,130,135,145,118,122,150,128,115),
  Respiration = c(16,18,20,22,26,17,19,24,21,15),
  Temperature = c(36.8,37.1,37.5,38.2,39.1,36.9,37.3,38.8,37.6,36.7)
)

print(hospital_data)

```

```
hospital_data$Status <- ifelse(
  hospital_data$Heart_Rate > 120 | hospital_data$Oxygen_Saturation < 90,
  "Critical",
  ifelse(hospital_data$Heart_Rate > 100,
    "Warning",
    "Normal")
)
```

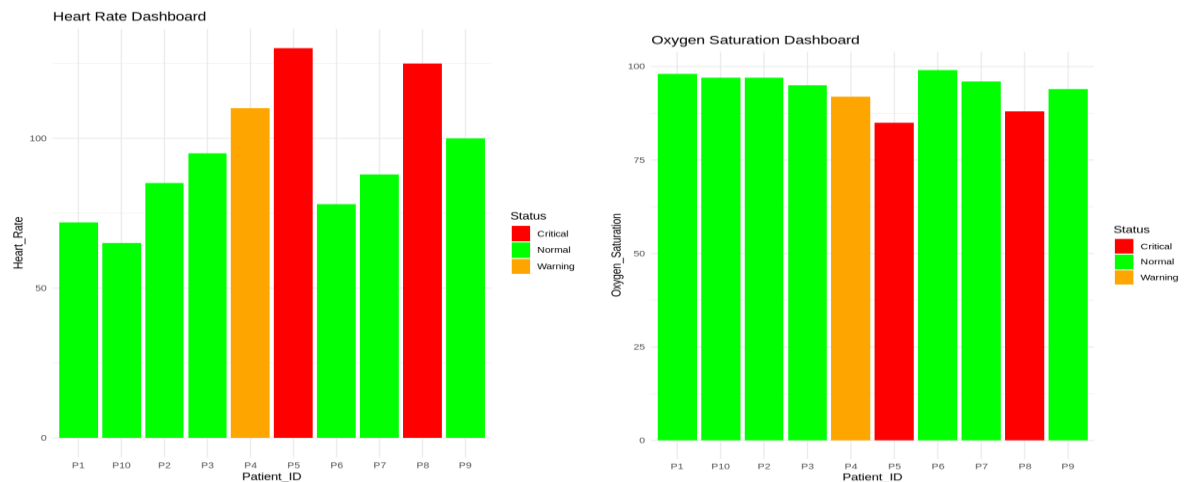
```
# Heart Rate Dashboard
```

```
ggplot(hospital_data,
  aes(Patient_ID, Heart_Rate, fill=Status)) +
  geom_col() +
  scale_fill_manual(values=c(
    "Normal"="green",
    "Warning"="orange",
    "Critical"="red")) +
  labs(title="Heart Rate Dashboard") +
  theme_minimal()
```

```
# Oxygen Saturation Dashboard
```

```
ggplot(hospital_data,
  aes(Patient_ID, Oxygen_Saturation, fill=Status)) +
  geom_col() +
  scale_fill_manual(values=c(
    "Normal"="green",
    "Warning"="orange",
    "Critical"="red")) +
  labs(title="Oxygen Saturation Dashboard") +
  theme_minimal()
```

Output:



Scenario-05: Global Climate Change Study

Python Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = pd.read_excel("scenario5.xlsx")

heatmap_data = data.pivot(index="Country",
                            columns="Year",
                            values="Temperature_Anomaly")

plt.figure(figsize=(8,6))

plt.imshow(heatmap_data,
           cmap="coolwarm",
           aspect="auto")

plt.colorbar(label="Temperature Anomaly (°C)")

plt.xticks(range(len(heatmap_data.columns)), heatmap_data.columns)

plt.yticks(range(len(heatmap_data.index)), heatmap_data.index)

plt.title("Heatmap using Coolwarm Palette")

plt.show()

plt.figure(figsize=(8,6))
```

```

plt.imshow(heatmap_data,
           cmap="RdBu_r",
           aspect="auto")

plt.colorbar(label="Temperature Anomaly (°C)")

plt.xticks(range(len(heatmap_data.columns)), heatmap_data.columns)

plt.yticks(range(len(heatmap_data.index)), heatmap_data.index)

plt.title("Heatmap using RdBu Palette")

plt.show()

plt.figure(figsize=(8,6))

plt.imshow(heatmap_data,
           cmap="PiYG",
           aspect="auto")

plt.colorbar(label="Temperature Anomaly (°C)")

plt.xticks(range(len(heatmap_data.columns)), heatmap_data.columns)

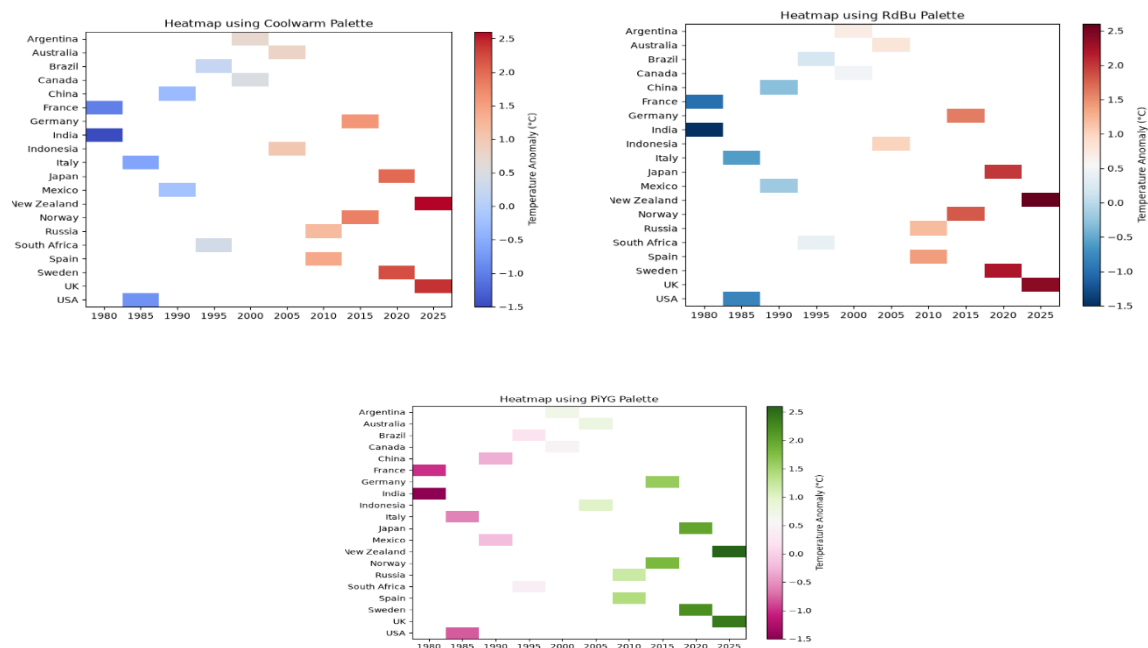
plt.yticks(range(len(heatmap_data.index)), heatmap_data.index)

plt.title("Heatmap using PiYG Palette")

plt.show()

```

Output:



R code:

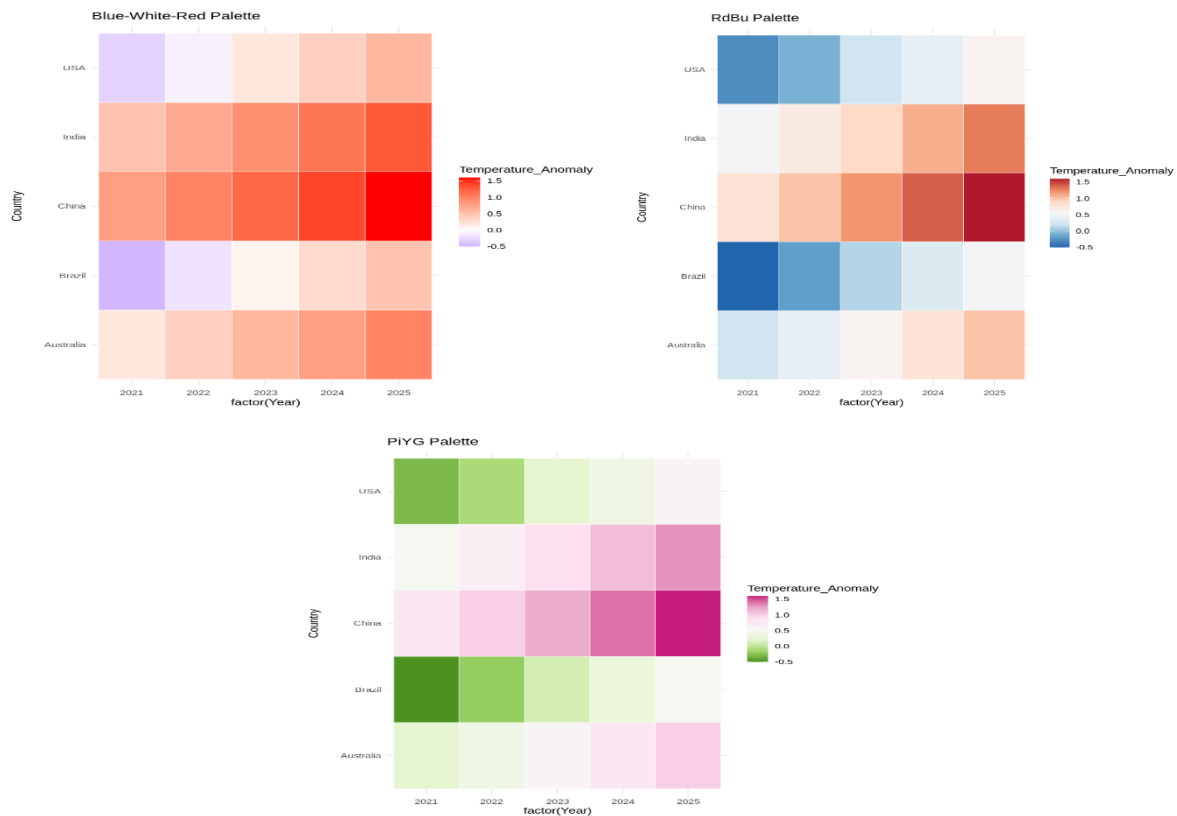
```
library(ggplot2)
library(RColorBrewer)
# Create Dataset
climate_data <- data.frame(
  Country = rep(c("India", "USA", "China", "Brazil", "Australia"), each=5),
  Year = rep(c(2021, 2022, 2023, 2024, 2025), 5),
  Temperature_Anomaly = c(
    0.5, 0.7, 0.9, 1.1, 1.3,
    -0.3, -0.1, 0.2, 0.4, 0.6,
    0.8, 1.0, 1.2, 1.4, 1.6,
    -0.5, -0.2, 0.1, 0.3, 0.5,
    0.2, 0.4, 0.6, 0.8, 1.0
  )
)
print(climate_data)
# Heatmap 1 - Blue White Red
ggplot(climate_data,
  aes(factor(Year), Country,
    fill=Temperature_Anomaly)) +
  geom_tile(color="white") +
  scale_fill_gradient2(low="blue",
    mid="white",
    high="red",
    midpoint=0) +
  labs(title="Blue-White-Red Palette") +
  theme_minimal()
# Heatmap 2 - RdBu
ggplot(climate_data,
  aes(factor(Year), Country,
```

```

    fill=Temperature_Anomaly)) +
  geom_tile(color="white") +
  scale_fill_distiller(
    palette="RdBu",
    direction=-1) +
  labs(title="RdBu Palette") +
  theme_minimal()
# Heatmap 3 - PiYG
ggplot(climate_data,
  aes(factor(Year), Country,
    fill=Temperature_Anomaly)) +
  geom_tile(color="white") +
  scale_fill_distiller(
    palette="PiYG") +
  labs(title="PiYG Palette") +
  theme_minimal()

```

Output:



Scenario-06: E-Commerce Sales Analytics

Python Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = pd.read_excel("scenario6.xlsx")

category_colors = {
    "Electronics": "blue",
    "Clothing": "green",
    "Furniture": "orange",
    "Grocery": "red"
}

plt.figure(figsize=(9,6))

for category in data["Category"].unique():
    subset = data[data["Category"] == category]
    plt.scatter(
        subset["Region"],
        subset["Sales"],
        s=subset["Sales"]/500,
        color=category_colors[category],
        alpha=0.7,
        edgecolors="black",
        label=category
    )

plt.title("Sales by Region and Product Category")
plt.xlabel("Region")
plt.ylabel("Sales")
plt.legend(title="Category")
plt.grid(True)
plt.show()

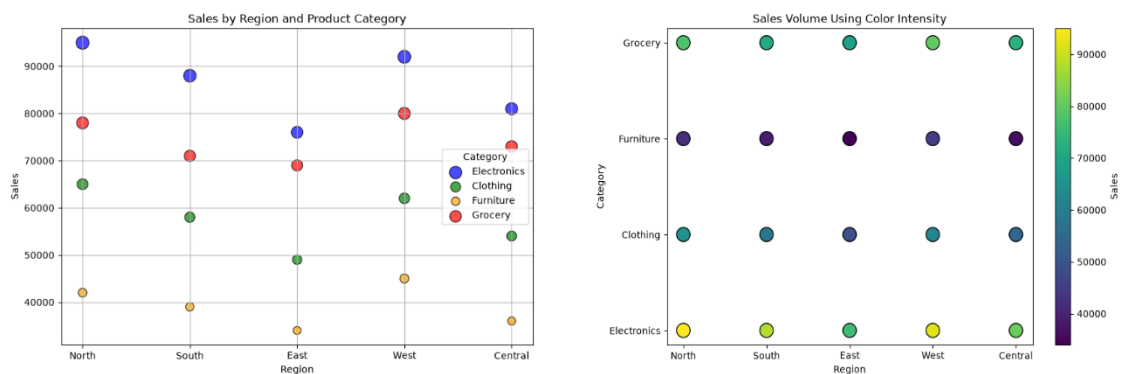
plt.figure(figsize=(9,6))
```

```

scatter = plt.scatter(
    data["Region"],
    data["Category"],
    c=data["Sales"],
    cmap="viridis",
    s=220,
    edgecolors="black"
)
plt.colorbar(scatter, label="Sales")
plt.title("Sales Volume Using Color Intensity")
plt.xlabel("Region")
plt.ylabel("Category")
plt.show()

```

Output:



R code:

```

library(ggplot2)

# Create Dataset
sales_data <- data.frame(
  Category = c("Electronics", "Clothing", "Furniture", "Groceries", "Sports",
    "Books", "Beauty", "Toys", "Jewelry", "Footwear"),
  Region = c("North", "South", "East", "West", "North",
    "South", "East", "West", "North", "South"),
  Sales = c(85000, 42000, 60000, 35000, 50000,

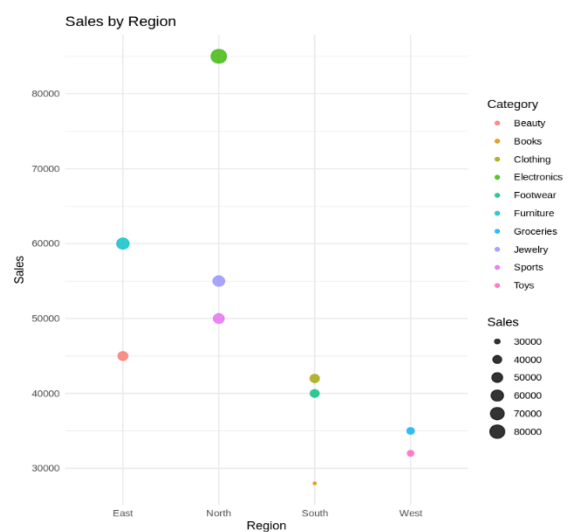
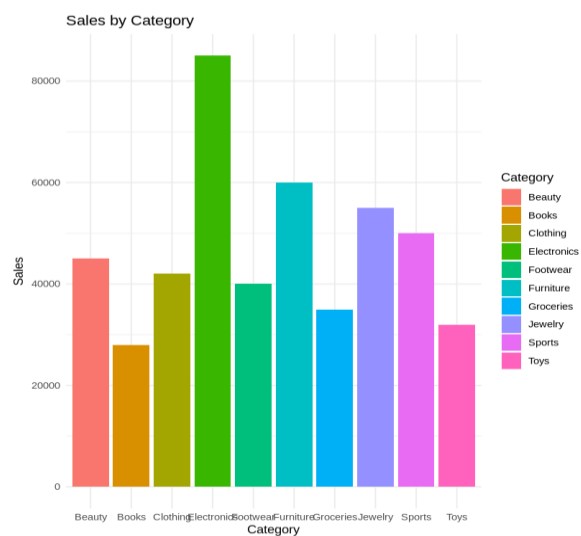
```

```

28000,45000,32000,55000,40000)
)
print(sales_data)
# Bar Chart
ggplot(sales_data,
       aes(Category, Sales, fill=Category)) +
  geom_col() +
  labs(title="Sales by Category") +
  theme_minimal()
# Bubble Chart
ggplot(sales_data,
       aes(Region, Sales,
           color=Category,
           size=Sales)) +
  geom_point(alpha=0.8) +
  labs(title="Sales by Region") +
  theme_minimal()

```

Output:



Scenario-07: Satellite Image Vegetation Analysis

Python Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = pd.read_excel("scenario7.xlsx")

plt.figure(figsize=(8,6))

scatter = plt.scatter(

    data["X"],

    data["Y"],

    c=data["NDVI"],

    cmap="Greens",

    s=180,

    edgecolors="black"

)

plt.colorbar(scatter, label="NDVI")

plt.title("NDVI using Sequential Palette (Greens)")

plt.xlabel("X Coordinate")

plt.ylabel("Y Coordinate")

plt.grid(True)

plt.show()

plt.figure(figsize=(8,6))

scatter = plt.scatter(

    data["X"],

    data["Y"],

    c=data["NDVI"],

    cmap="YlGn",

    s=180,

    edgecolors="black"

)

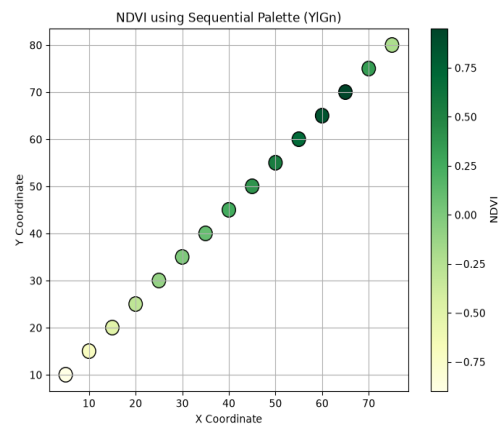
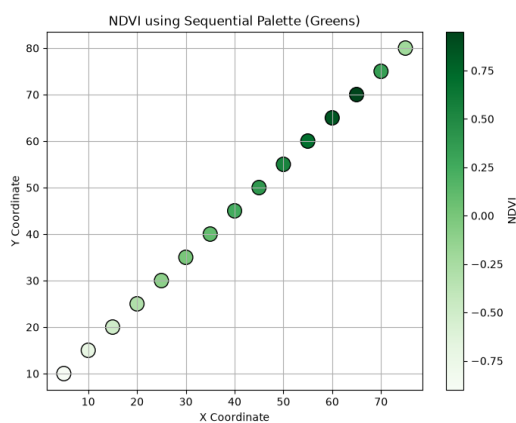
plt.colorbar(scatter, label="NDVI")
```

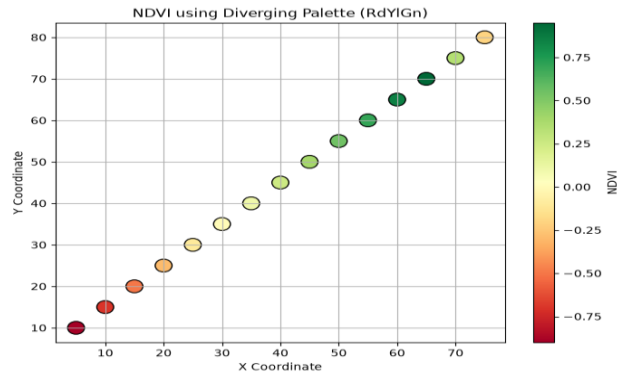
```

plt.title("NDVI using Sequential Palette (YlGn)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,6))
scatter = plt.scatter(
    data["X"],
    data["Y"],
    c=data["NDVI"],
    cmap="RdYlGn",
    s=180,
    edgecolors="black"
)
plt.colorbar(scatter, label="NDVI")
plt.title("NDVI using Diverging Palette (RdYlGn)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()

```

Output:





R code:

```
library(ggplot2)

# Create Dataset
ndvi_data <- data.frame(
  Location = paste("L",1:10,sep=""),
  NDVI = c(-0.8,-0.3,0.0,0.2,0.4,0.5,0.6,0.7,0.8,0.9)
)

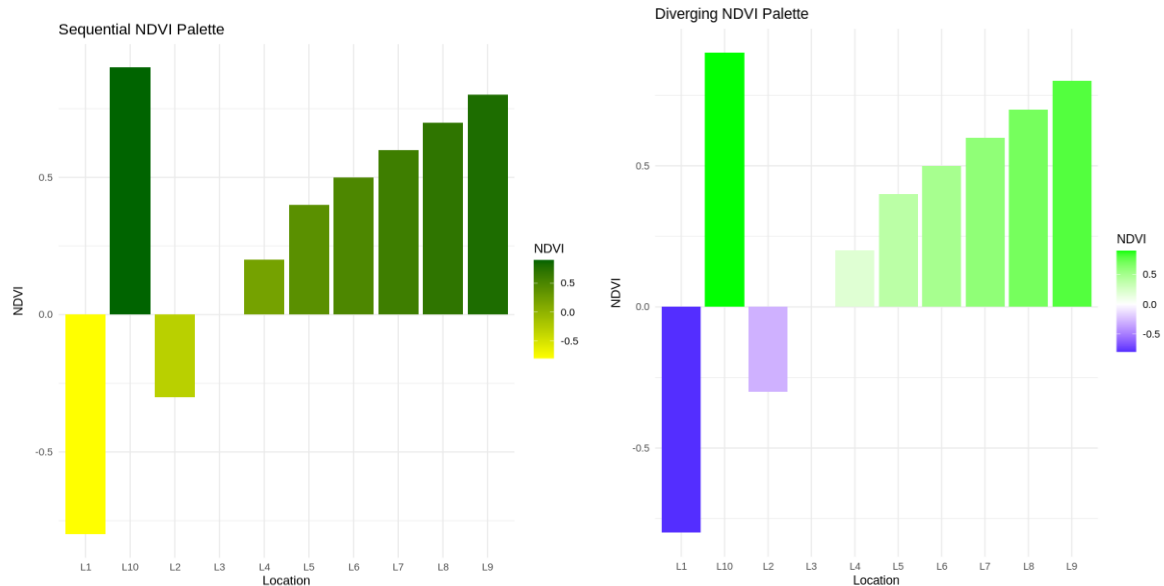
print(ndvi_data)

# Sequential Palette
ggplot(ndvi_data,
  aes(Location, NDVI, fill=NDVI)) +
  geom_col() +
  scale_fill_gradient(low="yellow",
    high="darkgreen") +
  labs(title="Sequential NDVI Palette") +
  theme_minimal()

# Diverging Palette
ggplot(ndvi_data,
  aes(Location, NDVI, fill=NDVI)) +
  geom_col() +
  scale_fill_gradient2(low="blue",
    mid="white",
    high="green",
```

```
midpoint=0) +
labs(title="Diverging NDVI Palette") +
theme_minimal()
```

Output:



Scenario-08: Cryptocurrency Market Volatility

Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_excel("scenario8.xlsx")

plt.figure(figsize=(10,6))

plt.plot(data["Date"], data["Bitcoin"], marker='o', color='orange', label='Bitcoin')
plt.plot(data["Date"], data["Ethereum"], marker='o', color='blue', label='Ethereum')
plt.plot(data["Date"], data["Solana"], marker='o', color='green', label='Solana')
plt.plot(data["Date"], data["Cardano"], marker='o', color='red', label='Cardano')

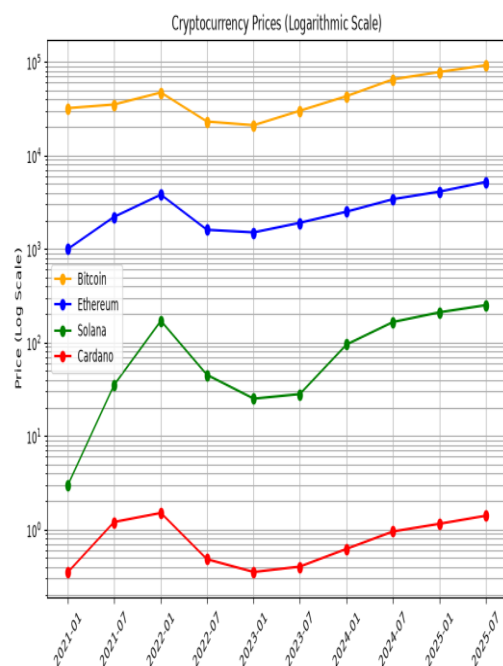
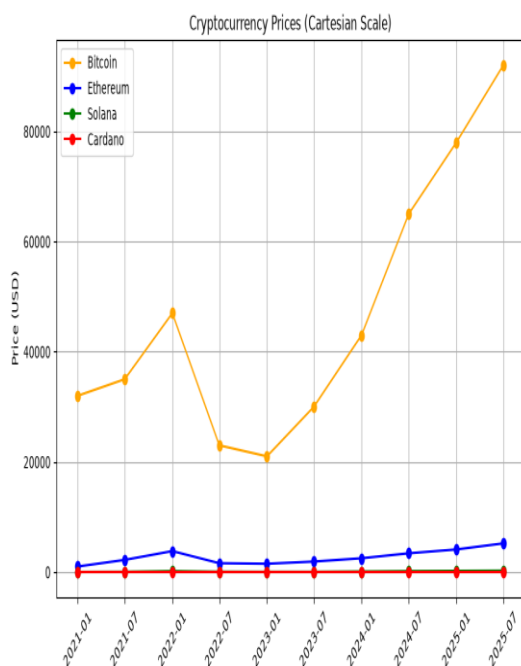
plt.title("Cryptocurrency Prices (Cartesian Scale)")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
```

```

plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(10,6))
plt.plot(data["Date"], data["Bitcoin"], marker='o', color='orange', label='Bitcoin')
plt.plot(data["Date"], data["Ethereum"], marker='o', color='blue', label='Ethereum')
plt.plot(data["Date"], data["Solana"], marker='o', color='green', label='Solana')
plt.plot(data["Date"], data["Cardano"], marker='o', color='red', label='Cardano')
plt.yscale('log')
plt.title("Cryptocurrency Prices (Logarithmic Scale)")
plt.xlabel("Date")
plt.ylabel("Price (Log Scale)")
plt.xticks(rotation=45)
plt.legend()
plt.grid(True, which="both")
plt.show()

```

Output:



R code:

```
library(ggplot2)

# Create Dataset

crypto_data <- data.frame(
  Day = 1:10,
  Bitcoin = c(60000,61000,62000,59000,64000,65000,67000,66000,68000,70000),
  Ethereum = c(3000,3100,3200,3050,3300,3400,3500,3450,3600,3700),
  Solana = c(120,125,130,128,140,145,150,148,155,160),
  Cardano = c(1.2,1.3,1.4,1.35,1.5,1.6,1.7,1.65,1.8,1.9)
)

print(crypto_data)

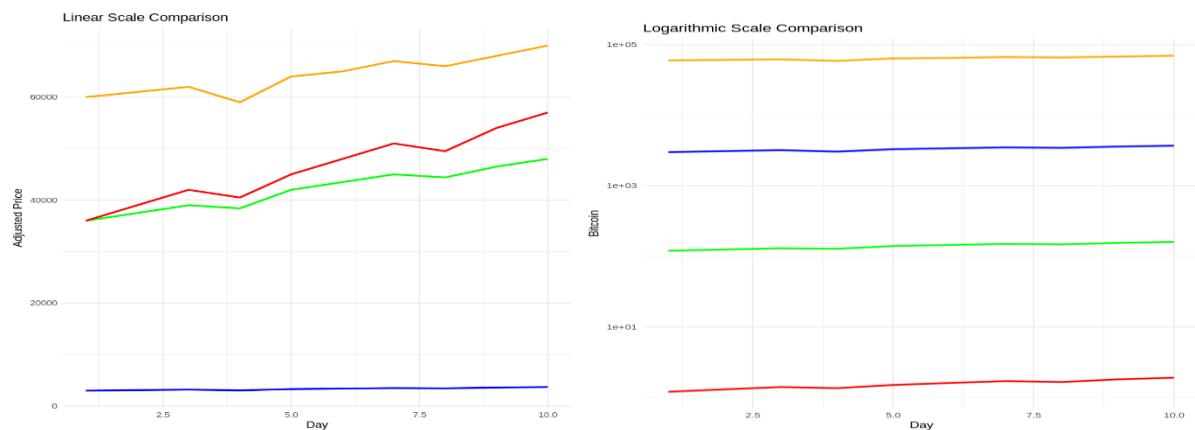
# Linear Plot

ggplot(crypto_data, aes(Day, Bitcoin)) +
  geom_line(color="orange", linewidth=1) +
  geom_line(aes(y=Ethereum), color="blue", linewidth=1) +
  geom_line(aes(y=Solana*300), color="green", linewidth=1) +
  geom_line(aes(y=Cardano*30000), color="red", linewidth=1) +
  labs(title="Linear Scale Comparison",
       y="Adjusted Price") +
  theme_minimal()

# Log Scale

ggplot(crypto_data, aes(Day, Bitcoin)) +
  geom_line(color="orange", linewidth=1) +
  geom_line(aes(y=Ethereum), color="blue", linewidth=1) +
  geom_line(aes(y=Solana), color="green", linewidth=1) +
  geom_line(aes(y=Cardano), color="red", linewidth=1) +
  scale_y_log10() +
  labs(title="Logarithmic Scale Comparison") +
  theme_minimal()
```

Output:



Scenario-09: Marathon Runner Performance Analysis

Python Code:

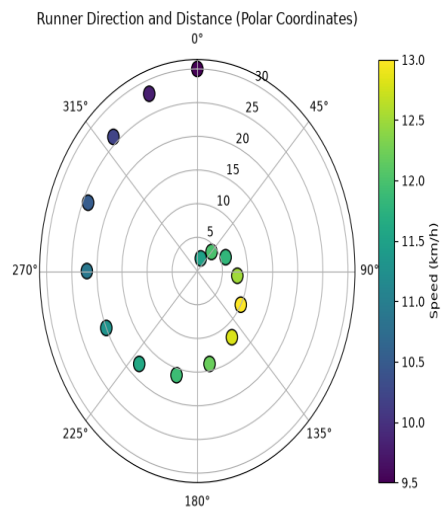
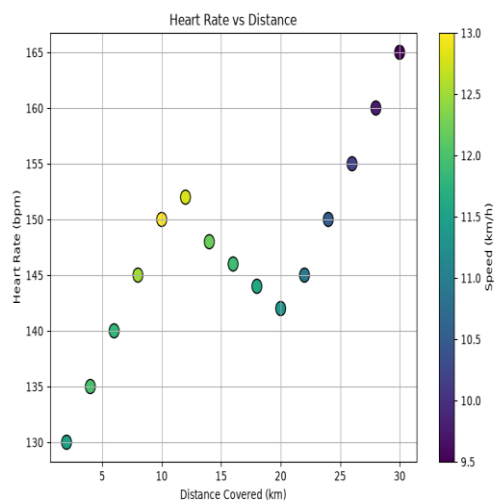
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data = pd.read_excel("scenario9.xlsx")
plt.figure(figsize=(9,6))
scatter = plt.scatter(
    data["Distance (km)"],
    data["Heart_Rate (bpm)"],
    c=data["Speed (km/h)"],
    cmap="viridis",
    s=120,
    edgecolors="black"
)
plt.colorbar(scatter, label="Speed (km/h)")
plt.title("Heart Rate vs Distance")
plt.xlabel("Distance Covered (km)")
plt.ylabel("Heart Rate (bpm)")
plt.grid(True)
plt.show()
```

```

theta = np.deg2rad(data["Direction (°)"])
r = data["Distance (km)"]
plt.figure(figsize=(8,8))
ax = plt.subplot(111, projection='polar')
scatter = ax.scatter(
    theta,
    r,
    c=data["Speed (km/h)"],
    cmap="viridis",
    s=120,
    edgecolors="black"
)
plt.colorbar(scatter, label="Speed (km/h)")
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
ax.set_title("Runner Direction and Distance (Polar Coordinates)")
plt.show()

```

Output:



R code:

```
library(ggplot2)

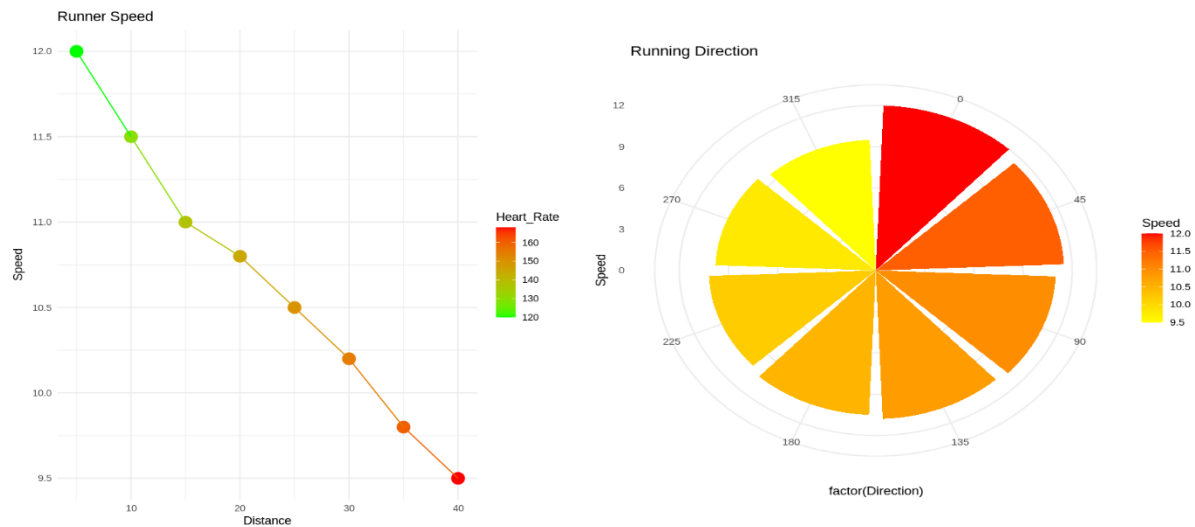
# Create Dataset
runner_data <- data.frame(
  Distance = c(5,10,15,20,25,30,35,40),
  Speed = c(12,11.5,11,10.8,10.5,10.2,9.8,9.5),
  Heart_Rate = c(120,130,138,145,150,155,160,168),
  Direction = c(0,45,90,135,180,225,270,315)
)

print(runner_data)

# Cartesian Plot
ggplot(runner_data,
  aes(Distance, Speed,
    color=Heart_Rate)) +
  geom_point(size=5) +
  geom_line() +
  scale_color_gradient(low="green",
    high="red") +
  labs(title="Runner Speed") +
  theme_minimal()

# Polar Plot
ggplot(runner_data,
  aes(factor(Direction),
    Speed,
    fill=Speed)) +
  geom_bar(stat="identity") +
  coord_polar() +
  scale_fill_gradient(low="yellow", high="red") +
  labs(title="Running Direction") +
  theme_minimal()
```

Output:



Scenario-10: COVID-19 Vaccination Campaign Dashboard

Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_excel("scenario10.xlsx")
plt.figure(figsize=(9,6))
scatter = plt.scatter(
    data["District"],
    data["Age_Group"],
    c=data["Vaccination_Rate"],
    cmap="YlGn",
    s=250,
    edgecolors="black"
)
plt.colorbar(scatter, label="Vaccination Rate (%)")
plt.title("Vaccination Dashboard - Policymakers")
plt.xlabel("District")
plt.ylabel("Age Group")
plt.show()

def get_color(rate):
```

```

if rate >= 90:
    return "green"
elif rate >= 80:
    return "gold"
else:
    return "red"
colors = data["Vaccination_Rate"].apply(get_color)
plt.figure(figsize=(9,6))
plt.scatter(
    data["District"],
    data["Age_Group"],
    c=colors,
    s=250,
    edgecolors="black"
)
plt.title("Vaccination Dashboard - General Public")
plt.xlabel("District")
plt.ylabel("Age Group")
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0],[0], marker='o', color='w', label='High ( $\geq 90\%$ )',
           markerfacecolor='green', markersize=10),

    Line2D([0],[0], marker='o', color='w', label='Medium (80-89%)',
           markerfacecolor='gold', markersize=10),

    Line2D([0],[0], marker='o', color='w', label='Low (<80%)',
           markerfacecolor='red', markersize=10)
]
plt.legend(handles=legend_elements)

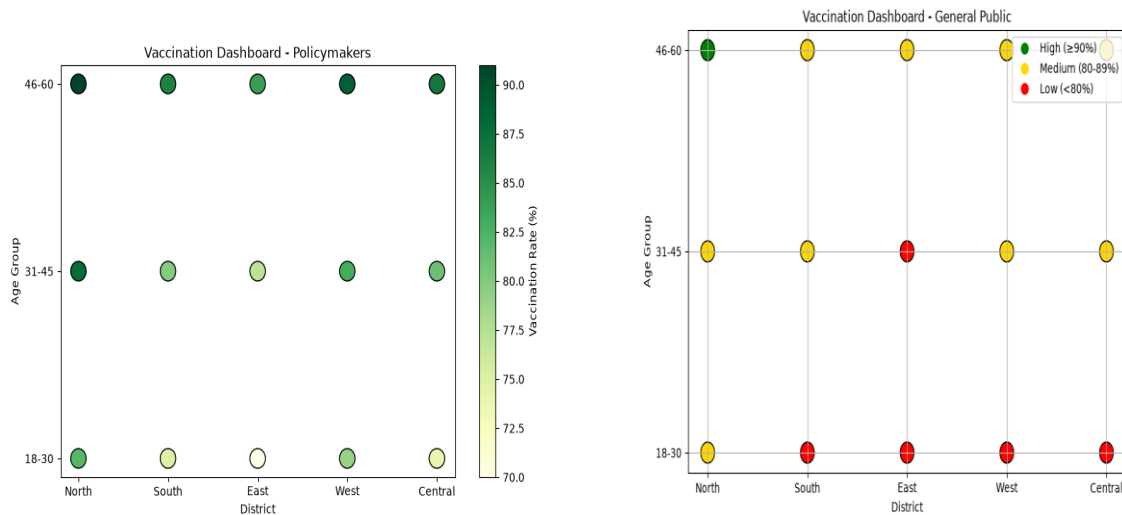
```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.show()
```

Output:



R code:

```
library(ggplot2)
```

```
# Create Dataset
```

```
vaccination_data <- data.frame(
```

```
  District = c("A","B","C","D","E","F","G","H"),
```

```
  Vaccination_Rate = c(92,84,76,68,95,82,74,88),
```

```
  Vaccine_Type = c("Covaxin","Covishield","Covaxin","Covishield",
```

```
    "Covaxin","Covishield","Covaxin","Covishield")
```

```
)
```

```
print(vaccination_data)
```

```
# Version 1
```

```
ggplot(vaccination_data,
```

```
  aes(District,
```

```
    Vaccination_Rate,
```

```
    fill=Vaccination_Rate)) +
```

```

geom_col() +
scale_fill_gradient(low="yellow",
                    high="green") +
labs(title="Vaccination Rates") +
theme_minimal()
# Version 2
ggplot(vaccination_data,
       aes(District,
           Vaccination_Rate,
           fill=Vaccine_Type)) +
geom_col() +
scale_fill_brewer(palette="Set2") +
labs(title="Vaccination by Vaccine Type") +
theme_minimal()

```

Output:

